תנאי שימוש בקובץ הדיגיטלי:

1. הקובץ הוא לשימושך **האישי** בלבד. פרטים מזהים שלך מוטבעים בקובץ בצורה גלויה ובצורה סמויה.

2. השימוש בקובץ הוא אך ורק למטרות לימוד, עיון ומחקר אישי.

3. העתקה או שימוש בתכנים נבחרים מותרת בהיקף העומד בכללי השימוש ההוגן, המפורטים בסעיף 19 לחוק זכות יוצרים 2007. במקרה של שימוש כאמור חלה חובה לציין את מקור הפרסום.

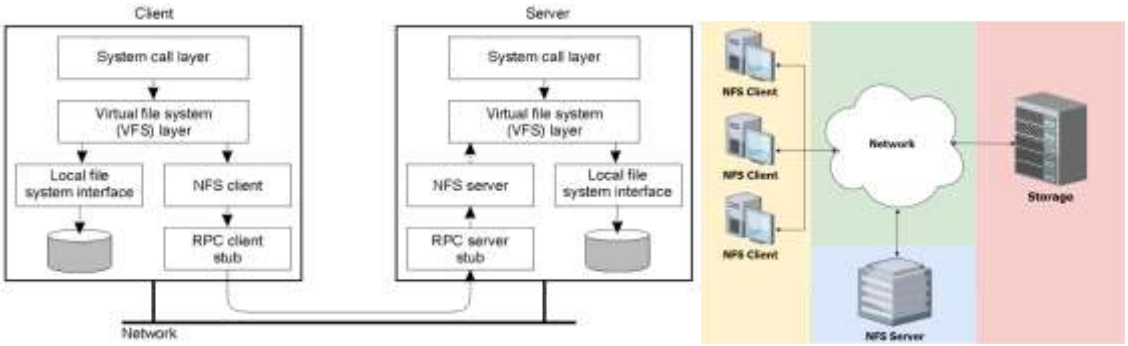
4. הנך רשאי/ת להדפיס דפים מחומר הלימוד לצורכי לימוד, מחקר ועיון אישיים. אין להפיץ או למכור תדפיסים כלשהם מתוך חומר הלימוד.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005997 belongs to Yair Kurtzman- do not distribute

מערכות קבצים מבוזרות (NFS)

מערכת קבצים מבוזרת היא מערכת קבצים שמבחינת הגישה מצד אחד נראית כמערכת קבצים רגילה; ומצד השני לא נשמרת במחשב אחד, אלא חלקים שלה מבוזרים (מתפרסים) על פני מספר מחשבים (שרתים).



לקבוצת מחשבים שמחוברים לרשת תקשורת מאפיינים שונים מאשר למחשב יחיד:

- רשת התקשורת עלולה להיות איטית משמעותית ביחס למהירות ערוצי תקשורת בתוך המחשב (כמו זיכרון לדיסק).
- במחשב כל הרכיבים פועלים/כבויים יחד (לרוב), ואילו ברשת חלק מהרכיבים יכולים לפעול בעוד אחרים ייכבו או יפלו.
- במחשב יחיד הגנת החומרה על ידי המערכת היא מוחלטת; ואילו ברשת מגיעות מנות ממקור חיצוני, ולכן מחשב ברשת לא נותן אמון מלא במנות מהרשת.

כל מחשב ברשת לרוב רץ עם מערכת משלו, והמחשבים מספקים שירותים זה לזה. מערכת זו קרויה "מערכת מבוזרת".

יתרונות הגישה לקבצים ברשת:

- קל לשתף נתונים בין משתמשים ותוכניות של מחשבים שונים, ובכך מתאפשר פיזור עומס חישובי.
- משתמש יכול לגשת לקבציו ממחשבים רבים שונים, אפילו ממקומות גיאוגרפיים שונים.
- ניהול שרת קבצים מרכזי/צביר שרתים קל מניהול נתונים על מספר רב של תחנות עבודה. למשל, וידוא גיבויים.
- שימוש מושכל בשרת מביא לחיסכון בנפח אחסון:
  - קובצי תוכניות ונתוני תוכניות רבים משוכפלים על כל מחשב שבו מותקנת התוכנית, ושיתוף במקום אחד חוסך מקום.
  - השרת מקצה לכל תחנה רק את הנפח שהיא זקוקה לו.

1. התייחסות לקבצים מרוחקים ופענוח שמם

נציג שתי גישות להתייחסות לקבצים מרוחקים:

- הצבת מערכת הקבצים המרוחקת במרחב השמות המקומי.
- מתן שמות לקבצים מרוחקים – השמות משקפים את שם השרת ואת שם הקובץ במרחב השמות של השרת.

1.1 הצבה שרירותית של מערכות קבצים מרוחקות

לפי הגישה הראשונה השרת מייצא את מערכת הקבצים שלו / תת-מרחב ששורשו מדריך בשרת. כל לקוח יכול להציב *mount* מערכת זו אצלו. יוניקס ולינוקס משתמשות בשיטה זו כשפרוטוקול הגישה למערכות הקבצים המרוחקות הוא *NFS*, בחלונות – בפרוטוקול *SMB* ועוד כמה פחות נפוצים.

היתרון של גישה זו: יכולת הצבת המערכת המרוחקת בלקוח שישקף את השימוש (למשל, הצבה ל- */home/students/cs*).

החיסרון של גישה זו: לאותו קובץ עלולים להיות שמות שונים במחשבים שונים, תכונה שעשוי הלגרום בלבול ובעיה אם רוצים לאחד מערכות כאלה.

1.2 שיקוף שמות שרתים במרחב השמות

לפי הגישה השנייה שם הקובץ מכיל את שם השרת; למשל, גישה לקבצים על שרתי *SMB* בחלונות ניתנת על ידי שרשור \\ בתחילית לשם השרת, ושרשור של זה לשם מערכת הקבצים שהשרת מייצא. למשל, השרת *main* מייצא את *c:* ל-*home*\, וגישה אליו *\\main\home* היא למעשה גישה ל- *c:* בשרת.

היתרון העיקרי של גישה זו לשיקוף: שמות הקבצים קבועים אצל כל הלקוחות.
יתרון משני: הצבה במרחב שמות מקומי דורש הרשאות של מנהל המערכת, בעוד בשיקוף אין דרישה כזאת.

החיסרון העיקרי של גישה זו: שמות הקבצים ארוכים ואינם משקפים את השימוש בהם.
ניתן להתגבר על חיסרון זה עם מצביעים סימבוליים. לא תמיד שיקוף של מיקום פיסי הוא חיסרון, כיוון שכך המשתמש מודע למיקום הפיסי של הקבצים גם במקרה שינותק.
חיסרון נוסף: העברת תתי-מרחבים משרת לשרת יכולה לשנות שמות של קבצים.

1.3 פענוח שמות קבצים מרוחקים

בפתיחת קובץ מרוחק המערכת צריכה לפענח את שמו למזהה פנימי שיאפשר גישה. הפענוח מתחיל כמו קובץ מקומי, וכשמגיע לתחילית שמייצגת נקודת הצבה של מערכת מרוחקת / תחילית כללית של מערכת מרוחקת, הפענוח עובר למודול תוכנה שמטפל בקבצים מרוחקים בפרוטוקול המתאים. למשל, אם הפרוטוקול הוא *NFS*, הפענוח יעבור למודול שמטפל ב- *NFS*, הפענוח יועבר לשרת, והשרת יחזיר מזהה פנימי ללקוח, שונה לרוב מהמזהה

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005997 belongs to Yair Kurtzman- do not distribute

הפנימי של הקובץ בשרת (למשל, *inode*). בגישות הבאות לקובץ הלקוח ישלח לשרת מזהה זה.

2. הטמנת נתונים והסמנטיקה של גישה לקבצים

אם כל גישה לקובץ מרוחק תתבסס על תקשורת עם השרת, הביצועים יהיו גרועים מאלה של מערכת קבצים מקומית:

- תקשורת איטית לעומת גישה מהירה לדיסק מקומי.
- עומס על השרת כשמשרת כמה לקוחות. גישה דרך רשת תהיה לרוב איטית מגישה מקומית, אך לא בסדר גודל.

פתרון הבעיות של עומס על השרת ותקשורת איטית: *caching* זמני בלקוח המונע צורך בתקשורת לכל פעולה על קובץ מרוחק.

בעיה: שינוי הסמנטיקה של גישה לקבצים לעומת גישה למערכת קבצים מקומית. הקבצים מהווים זיכרון משותף למחשבים רבים.

סמנטיקת הגישה לקבצים מקומיים ביוניקס/לינוקס:

- פעולות קריאה/כתיבה הן אטומיות. משמעות: העברת נתונים אל/מקובץ נראית כאילו נעשית בבת אחת ולא בהדרגה.
- קריאה רואה את כל הכתיבות שנעשו לפניה.
- פעולות מסוימות גורמות לשמירת נתונים במדיה יציבה. למשל, סגירת קובץ: קריאת המערכת חוזרת רק אחרי סיום כתיבת הנתונים בדיסק.

במערכת קבצים מבוזרת עם הטמנת נתונים תכונות אלו לא נשמרות. אם נתונים שנכתבים למטמון לא נשלחים אטומית לשרת, הקריאה והכתיבה לא יהיו פעולות אטומיות. לעתים מוותרים על תכונות אלו כדי שהביצועים יהיו גבוהים. יש מערכות שממשק שרת-לקוח הוא ברמת בלוקים בדיסק, ולשרת אין ידע על אירועים ברמת קריאות מערכת על קובץ אצל הלקוח, אלא רק פעולות על בלוקים בדיסק, שאינן מאפשרות לכמה לקוחות לגשת לאותם נתונים. קיצוניות שנייה: אי-הטמנה מקומית, ביצועים איטיים עקב רשת איטית ועומס על השרת.

פתרון מערכת *NFS*: מערכת הקבצים המבוזרת היא הנפוצה ביותר ביוניקס/לינוקס. הלקוח שומר מטמון בלוקים בגודל קבוע (לרוב) *8KB* מתוך קבצים. ב-*NFS* לא מובטחת אטומיות כי המעבר בין שרת ללקוח מתבצע בבלוקים בגודל קבוע, ולא מובטחת עדכניות של הנתונים אצל הלקוח. למרות הסמנטיקה החלשה ביצועיה טובים בגלל הטמנה אגרסיבית. כשרוב השימוש מרוחק מלקוחות מבודדים, בעיות הסמנטיקה לא משפיעות. עבודת כמה לקוחות על קבצים משותפים נכשלת בגלל הסמנטיקה החלשה. מערכת *NFS* מתגברת על כך בדרכים שונות.

3. הפרוטוקול חסר המצב של NFS

זהו פרוטוקול מיוחד בין לקוחות לשרת כדי למנוע מנפילת מחשבים להשפיע על אחרים.

המטרה: נפילת השרת לא תשפיע על הלקוחות, אם הוא חוזר לעבוד תוך זמן קצר, גם אם "שכח" את כל המידע על הלקוחות שהשתמשו בו לפני הנפילה.

פרוטוקול חסר מצב (*stateless*): השרת עונה לכל בקשה באופן נפרד מבלי לשמור מידע על לקוחות וקבצים פתוחים. כנ"ל הלקוח על השרת.

בעת בקשת קובץ מהשרת, הוא מחזיר ללקוח את ה- *inode* שלו כמזהה (לא שומר טבלת קבצים פתוחים ללקוחות) ואת זמן יצירת הקובץ. תהליך בלקוח מתרחש ממטמון, אלא אם זו קריאת נתונים שלא במטמון. כשלקוח מבקש להעביר קובץ מהמטמון לשרת, הוא מעביר את מזהה הקובץ, זיהוי משתמש ומיקום הבלוק המועבר בקובץ. השרת מחזיר אישור המבטיח שהנתונים הועברו למדיה יציבה בשרת, ואז הלקוח יכול למחוק אותם מהמטמון.

בסגירת קובץ: כל הבלוקים מועברים ממטמון הלקוח לשרת, ואין צורך להודיע לשרת על סגירה (כי הוא לא מחזיק מידע על קבצים פתוחים). כך גם על בקשות אחרות, כמו שינויים במרחב השמות, שינוי הרשאות וכו'.

כשלא מתקבל אישור מהשרת, לא ידוע ללקוח אם בקשתו בוצעה (ייתכן שבוצעה; ייתכן שמתעכבת ברשת; ייתכן שבוצעה, אך השרת נפל לפני אישורה). לכן אי קבלת אישור במשך זמן מה מצריכה שליחה מחודשת של הבקשה. שרת שלא יודע אם עשה פעולה זו או לא יבצע את הפעולה וישלח אישור. לכן רוב בקשות השירות יוגדרו בצורה אידמפוטנטית (ביצוע אותה פעולה מספר פעמים מניב אותה תוצאה). לא כל הפעולות יכולות להתבצע באופן אידמפוטנטי, ויש לקחת זאת בחשבון.

עוד הבדל מול סמנטיקת מערכת מקומית: קובץ נמחק רק אחרי שאין אליו הצבעות ואינו פתוח. יכול להיות מצב שבו קובץ פתוח בלקוחות אך נמחק בשרת, וניסיון גישת תהליך שבו קובץ פתוח ייכשל.

העובדה שהשרת חייב לכתוב נתונים למדיה יציבה לפני שליחת אישור עלולה להביא לעיכוב הלקוח הממתין לאישור. לכך הוספו שתי בקשות שירות: **כתיבה א-סינכרונית** (לא חייבת לכתוב מייד לדיסק), **ופעולת *commit*** שגורמת לשרת לכתוב לדיסק את כל פעולות הלקוח שבמטמון השרת.

אישור ה-*commit* כולל זהות כל הבלוקים שנכתבו. לקוח יבצע *commint* כשיסגור את הקובץ או כשהמטמון יתמלא. ייתכן שהשרת ייפול בין קבלת חלק מבקשות הכתיבה ובין ה-*commit*, ובקשות אלו יאבדו מהמטמון. הלקוח יזהה בהיעדר אישורים מהשרת שחלק מהכתיבות אבדו וישלח בלוקים אלו שוב.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005997 belongs to Yair Kurtzman- do not distribute

- ב- *hard mount* כששרת ***NFS*** לא מגיב לבקשות וכאשר מערכת הקבצים מוצבת אצל הלקוח, הלקוח ישלח את הבקשות שוב ושוב עד מענה.
- ב-*soft mount* יהיה מספר קבוע של חזרות על הבקשה, ואז תישלח שגיאה לתהליך המבוקש.

בהצבה קשיחה, כשהתהליך נתקע עד מענה השרת, אפשר רק 'להרוג' את התהליך, אך לא ניתן לעצור את הבקשה. הצבה רכה עלולה לגרום לקריאות מערכת להיכשל כשיש עומס. לכן צורת ההצבה לנתונים שניגשים אליהם אינטראקטיבית חייבת להיות ניתנת לפסיקה.

הצבה קשיחה טובה למערכות קבצים שהמחשב לא יכול להסתדר בלעדיהם או למערכות לא אינטראקטיביות.

4. קונסיסטנטיות בעזרת "חוזים" קצרי מועד

חוזים קצרי מועד (*short term leases*) מאפשרים גישה קונסיסטנטית לקבצי *NFS* מכמה לקוחות בלי לפגוע בשרידות של *NFS* . השרת יכול להורות ללקוחות לא להטמין נתונים בקבצים שבשימוש כמה לקוחות. במקרה כזה השרת לא חסר מצב, והוא שומר מידע על פתיחת קבצים. לקוח יכול לגשת לקובץ רק אם יש לו חוזה תקף לקובץ ולסוג הגישה.

קיימים שלושה סוגי חוזים:

- **חוזה קריאה עם הטמנה**: מרשה קריאה בלבד והטמנה אצל הלקוח. הוא שימושי כאשר כמה לקוחות קוראים בלבד ולא משנים פרטים בחוזה.
- **חוזה כתיבה עם הטמנה**: מאפשר קריאה וכתיבה והטמנת נתונים אצל הלקוח. הוא שימושי כאשר לקוח אחד משתמש בקובץ.
- **חוזה ללא הטמנה**: אסור ללקוח להטמין, כל קריאה/כתיבה חייבת להיות מועברת לשרת. הוא שימושי כאשר כמה לקוחות קוראים את הקובץ וגם כותבים בו.

רוב חוזי העבודה הם משני הסוגים הראשונים, ולכן תהיה הטמנה והביצועים יהיו טובים. לקוח שרוצה לקרוא מבקש חוזה קריאה, ואם אין חוזי כתיבה מקבל חוזה קריאה עם הטמנה. מספר לקוחות יכולים להחזיק חוזה כזה על אותו קובץ. לקוח שרוצה לכתוב מבקש חוזה כתיבה, ואם אין חוזים על הקובץ הוא מקבל כתיבה עם הטמנה, אחרת יחליף השרת לכל הלקוחות את החוזים לחוזי כתיבה ללא הטמנה, ולאחר אישור ההחלפה מכולם יעניק כזה גם למבקש הכתיבה.

אם שרת רוצה לשנות ללקוח חוזה אך הלקוח לא עונה, אז אם הלקוח נפל, כשיעלה לא יזכור את החוזה ואין בעיה; אם הוא רק לא יצר קשר זמן רב, אך עדיין מחזיק בחוזה, עלולה להיות בעיה עם הקובץ. לכן כל החוזים הם קצרי מועד ופגים תוך זמן קצר (חצי דקה בערך), ולקוח שרוצה להמשיך להשתמש צריך לחדש את החוזה.

אם השרת לא מסכים להאריך חוזה עם הטמנה, הלקוח יעתיק את ההטמנה לשרת מהר ככל האפשר. אם השרת לא מצליח לשנות חוזה ללקוח בעקבות היעדר מענה, הוא ממתין

לתפוגת החוזה, וממשיך בידיעה שהלקוח כבר ללא החוזה. השרת מחכה זמן מה אחרי הפקיעה, למקרה שיקבל נתונים מהלקוח אחרי פקיעת החוזה שלו, כדי להיות בטוח שהחוזה פקע גם לפי שעון הלקוח בנוסף לזמן הגעת הנתונים. אם שרת נפל ולא יודע אילו חוזים בתוקף אצל הלקוחות (כי לא קיבל עדכונים), הוא ממתין דקה-שתיים שכל החוזים יפוגו, ואז יכול להתחיל לחלק חוזים חדשים ללקוחות. גם במנגנון זה נשמר מידע על לקוחות ושרתים, אך לאובדן המידע אין השפעות ארוכות טווח ולכן שרידות המערכת לא נפגעת.

5. אבטחת מידע במערכות קבצים מבוזרות

בבקשות לקוח השרת צריך להחליט אם להיענות או לסרב, על בסיס המידע מי המחשב המבקש ומי המשתמש המבקש.

גישה אחת: לוודא שהמחשב מריץ מערכת הפעלה אמינה בעיני השרת, והיא אחראית על וידוא זהות המשתמשים, כך פועלת *NFS*. השרת מחזיק רשימת כתובות *IP* של מחשבים מורשים כנ"ל ושל פורטים מורשים, רק בקשות משם ייענו. הנחת השרת לא מתקיימת אם גנבי מידע שולחים מכתובות *IP* מורשות.

גישה שנייה : גישה מתוחכמת יותר, המבוססת על זיהוי המשתמש על ידי השרת. המשתמש צריך להזדהות מול השרת, למשל עם סיסמה, רק בתחילת העבודה מול השרת. השרת מעביר בתגובה ללקוח אישור גישה מספרי שקשה לזיוף, ובכל גישת לקוח לשרת מעביר לו אישור זה. כך השרת לא צריך לסמוך על מערכת ההפעלה של הלקוח.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman